

Contents

AI Coding CLI Tools: The Definitive Comparison (March 2026)	1
1. Executive Summary	2
2. Feature Comparison Matrix	2
Core Identity	2
Model & Context	2
Core Capabilities	3
Agent & Autonomy	3
Safety & Sandboxing	4
Extensibility & Ecosystem	4
Memory & Persistence	4
Integration & Access	5
3. Architecture Comparison	5
Key Architectural Differences	7
4. Model Comparison	7
Available Models and Specifications	7
Secondary/Budget Models	7
API Token Pricing (per million tokens)	7
5. Developer Experience	8
Setup & Onboarding	8
Daily Workflow Experience	8
6. Ecosystem	9
Skills, Plugins, and Extensions	9
MCP (Model Context Protocol) Ecosystem	9
7. Best For (Use Cases)	9
Decision Matrix	9
8. Benchmark Results	11
SWE-bench Verified (500 Python tasks, March 2026)	11
SWE-bench Pro (1,865 multi-language tasks)	11
SWE-CI (Continuous Integration, March 2026)	11
Real-World First-Pass Correctness (from developer reports)	11
9. Pricing Comparison	12
Subscription Plans	12
Cost Per Task (Estimated, API pricing)	12
Best Value by Profile	12
10. Verdict & Recommendations	13
The Clear Winner (Overall): Claude Code	13
The Best Free Option: Gemini CLI	13
The Safest Execution: Codex CLI	13
The Most Promising: Grok Build	13
The IDE Developer's Choice: Cursor	13
The Model-Agnostic Veteran: Aider	13
Quick Decision Guide	13
Methodology & Disclaimers	14
Sources	14

AI Coding CLI Tools: The Definitive Comparison (March 2026)

Claude Code vs. Gemini CLI vs. Codex CLI vs. Grok Build vs. Cursor vs. Aider

Last updated: March 25, 2026

1. Executive Summary

Claude Code (Anthropic) is the most mature terminal-based AI coding agent, launched February 2025 and now at v2.1.83. It runs Opus 4.6 with a 1M token context window and leads in autonomous correctness, multi-agent orchestration (Agent Teams, Background Agents), and developer experience (Skills, Hooks, Memory, Remote Control, Voice Mode). It is the tool that most consistently produces working code on the first attempt.

Gemini CLI (Google) is the open-source contender, launched June 2025 under Apache 2.0. It runs Gemini 3.1 Pro with a 1M token context window and differentiates on its genuinely useful free tier (1,000 requests/day), Google Search grounding, and growing sandbox/security features. It is the best option for budget-conscious developers and those already in the Google ecosystem.

Codex CLI (OpenAI) is the sandbox-first terminal agent, launched April 2025 and rewritten in Rust for speed and safety. It runs GPT-5.4 with up to 1M tokens (272K standard) and leads in execution safety with OS-native sandboxing (Seatbelt on macOS, Bubblewrap on Linux). It integrates naturally with the ChatGPT subscription ecosystem.

Grok Build (xAI) is the ambitious newcomer, announced January 2026 and still on a waitlist as of March 2026. Its headline feature is 8 parallel coding agents with an Arena Mode that ranks competing solutions algorithmically. It uses Grok Code Fast 1 with a 2M token context window at aggressive pricing, but availability remains limited.

Cursor (Anysphere) is the IDE-based alternative – a VS Code fork launched March 2023, now at version 2.0 with multi-agent Composer, Background Agents, BugBot, and deep codebase indexing. It is the best choice for developers who prefer a full IDE experience over a terminal workflow.

Aider (open source) is the community-driven veteran, launched mid-2023 by Paul Gauthier. It connects to 100+ models, has best-in-class git integration with automatic commits, and costs nothing beyond API fees. It is the most flexible and model-agnostic option available.

2. Feature Comparison Matrix

Core Identity

Feature	Claude Code	Gemini CLI	Codex CLI	Grok Build	Cursor	Aider
Launch Date	Feb 2025	Jun 2025	Apr 2025	Jan 2026 (waitlist)	Mar 2023	Mid-2023
Current Version	v2.1.83	v1.x	v0.111+ (Rust)	Pre-release	v2.0	v0.75+
Developer	Anthropic	Google	OpenAI	xAI	Anysphere	Paul Gauthier / Community
Open Source	Partial (source-available)	Yes (Apache 2.0)	Yes (Apache 2.0)	No	No	Yes (Apache 2.0)
Written In Interface	TypeScript/Node.js Terminal CLI	TypeScript Terminal CLI	Rust Terminal CLI	Unknown Terminal CLI + Web UI	Electron/TS VS Code fork (IDE)	Python Terminal CLI + Web UI

Model & Context

Feature	Claude Code	Gemini CLI	Codex CLI	Grok Build	Cursor	Aider
Base Model	Opus 4.6	Gemini 3.1 Pro	GPT-5.4	Grok Code Fast 1 / Grok 4 Fast	Multi-model (GPT-5, Claude, Gemini)	Model-agnostic (100+ models)
Context Window	1M tokens	1M tokens	272K standard / 1M experimental	2M tokens	Varies by model	Varies by model
Model Switching	Yes (Haiku/Sonnet/Opus)	Yes (Flash/Pro)	Yes (codex-mini/GPT-5.4)	Yes (Code Fast/Grok 4 Fast)	Yes (any frontier model)	Yes (any model via API)
Thinking/Reasoning	Yes (extended thinking)	Yes (thinking mode)	Yes (chain-of-thought)	Yes	Yes	Yes (model-dependent)

Core Capabilities

Feature	Claude Code	Gemini CLI	Codex CLI	Grok Build	Cursor	Aider
File Reading	Yes	Yes	Yes	Yes	Yes	Yes
File Editing	Yes (surgical edits)	Yes	Yes	Yes	Yes (multi-file Composer)	Yes (diff-based)
Bash/Shell Execution	Yes	Yes	Yes (sandboxed)	Yes	Yes (terminal panel)	Limited (run commands)
Web Browsing	Yes (WebSearch + WebFetch)	Yes (Google Search grounding)	Yes (web search cache + live)	Unknown	No (IDE-based)	Yes (Playwright scraping)
Image/Vision Support	Yes (screenshots, designs)	Yes	Yes (screenshots, designs)	Unknown	Yes (paste images)	Yes (image input)
Voice Input	Yes (/voice, 20 languages)	No	No	No	No	Yes (voice-to-code)
Computer Use	Yes (mouse, keyboard, screen)	No	Yes (GPT-5.4 native)	No	No	No

Agent & Autonomy

Feature	Claude Code	Gemini CLI	Codex CLI	Grok Build	Cursor	Aider
Subagents	Yes (Agent tool, background)	Partial (plan mode)	Yes (subagent workflows)	Yes (8 parallel agents)	Yes (background agents)	No
Multi-Agent	Yes (Agent Teams)	No	Yes (multi-agent)	Yes (8 concurrent + Arena Mode)	Yes (up to 8 agents, Composer)	No
Background Agents	Yes	No	Partial	Unknown	Yes (cloud-based)	No
Parallel Execution	Yes	No	Yes	Yes (core differentiator)	Yes	No

Feature	Claude Code	Gemini CLI	Codex CLI	Grok Build	Cursor	Aider
Scheduling/Cron (/loop with intervals)	Yes	No	No	No	No	No

Safety & Sandboxing

Feature	Claude Code	Gemini CLI	Codex CLI	Grok Build	Cursor	Aider
Sandboxing	Partial (permission prompts)	Yes (gVisor, LXC containers)	Yes (Seatbelt/Bubblewrap, strongest)	Unknown	N/A (IDE)	No
Permission System	Yes (allow/deny per tool)	Yes (approval modes)	Yes (read-only/workspace/full)	Unknown	Yes (approve changes)	Yes (approve edits)
Checkpoints/Undo (auto-checkpoints, /rewind)	Yes	No	Yes (session transcripts)	Unknown	Yes (git-based)	Yes (git commits per change)
Worktree Isolation	Yes (git worktrees)	No	No	Unknown	No	No

Extensibility & Ecosystem

Feature	Claude Code	Gemini CLI	Codex CLI	Grok Build	Cursor	Aider
MCP Support	Yes (client)	Yes (client)	Yes (client)	Unknown	Yes (client)	Partial (available as MCP server)
Skills/Extensions	Yes (SKILL.md system)	Partial (preview)	No formal system	No	No (uses rules files)	No
Hooks System	Yes (12 lifecycle events)	No	Partial (experimental)	No	No	No
Plugin Ecosystem	Yes (npm-based plugins)	Community extensions	No	No	VS Code extensions	No
Custom Commands	Yes (.claude/commands/)	Partial (aliases)	No	No	No	No

Memory & Persistence

Feature	Claude Code	Gemini CLI	Codex CLI	Grok Build	Cursor	Aider
Memory System	Yes (CLAUDE.md + MEMORY.md auto-memory)	Yes (GEMINI.md + save_memory tool)	Yes (SQLite-backed, cross-session)	Unknown	Yes (remembers past chats)	No native (manual context)
Session Persistence	Yes (resume sessions)	Yes (-resume flag, 30-day retention)	Yes (JSONL transcripts, resume subcommand)	Unknown	Yes	Partial (/save and /load)
Project Instructions	Yes (CLAUDE.md at multiple scopes)	Yes (GEMINI.md)	Yes (AGENTS.md / instructions)	Unknown	Yes (.cursorrules)	Yes (.aider.conf.yml)

Integration & Access

Feature	Claude Code	Gemini CLI	Codex CLI	Grok Build	Cursor	Aider
Git Integration	Yes (commits, diffs, branches)	Yes (basic)	Yes (basic)	Yes (GitHub integration)	Yes (BugBot PR reviews)	Yes (best-in-class: auto-commits)
IDE Integration	Yes (VS Code extension)	No	Yes (VS Code extension)	No	Native (is an IDE)	Yes (VS Code, Neovim, etc.)
Remote Control	Yes (claude.ai/code, mobile apps)	No	No	No	Yes (cloud agents)	No
SDK/Headless Mode	Yes (Agent SDK, JSONL I/O)	Partial (API mode)	Yes (headless mode)	Unknown	No	Yes (scripting mode)
Mobile Access	Yes (iOS/Android via Remote Control)	No	No	No	No	No

3. Architecture Comparison

AI CODING CLI ARCHITECTURE MAP

CLAUDE CODE (Anthropic)	GEMINI CLI (Google)
Terminal / VS Code / Mobile / Web Remote	Terminal
Agent Orchestrator Main Agent Background Agents Agent Teams Skill Dispatcher	Plan Mode Engine Task Decomposition Sequential Execution
Tools Layer File Edit/Read Bash Execution Web Search/Fetch Computer Use MCP Clients	Built-in Tools Google Search File Ops Shell Commands Web Fetch MCP / Extensions
Memory Layer CLAUDE.md (manual) MEMORY.md (auto) Checkpoints	Sandbox (gVisor / LXC) Gemini 3.1 Pro / Flash (1M tokens)
Hooks (12 events) Plugins (npm) Skills (SKILL.md)	Open Source (Apache 2.0) Written in TypeScript

Opus 4.6 (1M tokens)

Source-available
Written in TypeScript

CODEX CLI (OpenAI)

Terminal / VS Code

Rust Runtime Engine
Millisecond startup
Low memory footprint
Type-safe sandboxing

Approval Modes
Read-only
Workspace-write
Full access

OS-Native Sandbox
Seatbelt (macOS)
Bubblewrap (Linux)

Memory (SQLite-backed)
Sessions (JSONL)
MCP Client

GPT-5.4 (272K/1M)

Open Source (Apache 2.0)
Written in Rust

GROK BUILD (xAI)

Terminal / Web UI

Multi-Agent Orchestrator
Up to 8 Agents
Side-by-side output
Arena Mode (ranking)

Context Usage Tracker

GitHub Integration

Grok Code Fast 1

Grok 4 Fast
(2M tokens)

Closed source

Waitlist (as of Mar 2026)

CURSOR (Anysphere)

VS Code Fork (Electron)

Codebase Indexing Engine
(proprietary, deep)
Multi-file awareness
Cross-file relations

Composer 2.0
Multi-agent (8x)
Background agents
Proprietary model

BugBot (PR reviews)
Tab Autocomplete
Inline Edits

Multi-model Support
(GPT-5, Claude, Gemini)

Closed source
VS Code fork (Electron)

AIDER (Community)

Terminal / Web UI

Repo Map (git-aware)
Whole-repo context
Auto git commits

Edit Modes

Whole file
Diff-based
Architect mode

100+ Model Support

OpenAI, Anthropic
Google, Local LLMs
OpenRouter, etc.

Linting + Test Runner
Voice Input

Open Source (Apache 2.0)
Written in Python

Key Architectural Differences

Aspect	Approach
Claude Code	Deepest feature stack. Agent orchestration + memory + hooks + skills + plugins + remote control. Most “platform-like” of the CLIs.
Gemini CLI	Lean and open. Relies on Google Search grounding as a differentiator. Sandbox-first with gVisor/LXC.
Codex CLI	Community-driven extensions.
Grok Build	Performance-first (Rust). Security-first (OS-native sandboxing). Minimal overhead, maximum safety.
Cursor	Parallelism-first. 8 concurrent agents competing via Arena Mode is architecturally unique. Highest theoretical throughput.
Aider	IDE-first. Deep codebase indexing that understands cross-file relationships. Best for developers who want AI embedded in their editor.
	Model-agnostic, git-native. Maps your entire repo and commits every change. Lightest dependency footprint.

4. Model Comparison

Available Models and Specifications

Tool	Primary Model	Context Window	Speed (tokens/s)	SWE-bench Verified
Claude Code	Opus 4.6 (Thinking)	1M	~80-120	80.8%
Gemini CLI	Gemini 3.1 Pro	1M	~100-150	80.6%
Codex CLI	GPT-5.4	272K (1M exp.)	~90-130	78.2%
Grok Build	Grok Code Fast 1	2M	Unknown	70.8%
Cursor	Multi-model	Varies	Varies	N/A (tool, not model)
Aider	Multi-model	Varies	Varies	N/A (tool, not model)

Secondary/Budget Models

Tool	Budget Model	Context Window	Use Case
Claude Code	Haiku 3.5 / Sonnet 4	200K	Fast tasks, cost reduction
Gemini CLI	Gemini 3 Flash	1M	Free tier, lighter tasks
Codex CLI	GPT-5.4 Mini	128K	4x more usage, lighter tasks
Grok Build	Grok 4 Fast	2M	Alternative to Code Fast
Cursor	Any frontier model	Varies	User’s choice
Aider	Any model via API	Varies	User’s choice

API Token Pricing (per million tokens)

Model	Input	Output	Cache Read
Claude Opus 4.6	\$15.00	\$75.00	\$1.50
Claude Sonnet 4	\$3.00	\$15.00	\$0.30
Gemini 3.1 Pro	\$1.25	\$10.00	\$0.31
Gemini 3 Flash	\$0.075	\$0.30	\$0.01
GPT-5.4	\$2.50	\$10.00	\$0.63
GPT-5.4 Mini	\$0.40	\$1.60	\$0.10
Grok Code Fast 1	\$0.20	\$1.50	Free
Grok 4 Fast	\$0.20	\$0.50	Free

Note: Grok models offer the lowest per-token pricing and free cache reads, making them attractive for high-volume parallel agent workflows. Claude Opus 4.6 has the highest per-token cost but often requires fewer iterations, potentially lowering total cost per completed task.

5. Developer Experience

Setup & Onboarding

Aspect	Claude Code	Gemini CLI	Codex CLI	Grok Build	Cursor	Aider
Install Command	<code>npm install -g @anthropic-ai/claude-code</code>	<code>npm install @anthropic-ai/gemini-cli</code>	<code>brew install codex</code> (Rust binary)	Waitlist	Download .dmg / installer	<code>pip install aider-chat</code>
Time to First Prompt	~2 min	~1 min	~1 min	N/A	~5 min (IDE setup)	~2 min
Auth Method	Anthropic API key or Claude subscription	Google account (free) or API key	OpenAI API key or ChatGPT subscription	xAI API key	Subscription	Any API key
Zero-Cost Start	No (requires subscription or API key)	Yes (1,000 free requests/day)	No (requires subscription or API key)	No	Yes (limited free plan)	No (BYOK, API costs only)
Learning Curve	Medium (rich feature set)	Low (simple, familiar)	Low (clean, focused)	Unknown	Low (VS Code familiarity)	Low (simple commands)
Documentation Quality	Excellent (code.claude.com/docs)	Good (growing)	Good (developers.openai.com)	Minimal	Good (cursor.com/docs)	Excellent (aider.chat)

Daily Workflow Experience

Claude Code feels like having a senior engineer in your terminal. The Skills system means you can build reusable workflows (`/deploy`, `/review`, `/pipeline`) that compound over time. The Memory system learns your patterns. Remote Control means you can start a task on your laptop and monitor it from your phone. The `/loop` command turns it into a lightweight monitoring tool. The sheer breadth of features is unmatched, but it can take weeks to discover and master them all.

Gemini CLI feels clean and fast. Google Search grounding means it can look up current documentation without extra configuration. The free tier is genuinely usable for real work, not just a trial. Plan Mode helps it break down tasks systematically. The sandbox features (gVisor, LXC) provide real isolation for running untrusted commands. The tradeoff is fewer advanced orchestration features compared to Claude Code.

Codex CLI feels snappy – the Rust rewrite is noticeable. It starts instantly, uses minimal memory, and the OS-native sandboxing provides genuine peace of mind when running generated commands. The three approval modes

(read-only, workspace, full) are simple and intuitive. Web search integration is well-executed with both cached and live modes. The tradeoff is a thinner extension ecosystem.

Grok Build (based on early access reports) emphasizes parallelism. Seeing 8 agents working simultaneously on different approaches to the same problem is novel. The side-by-side comparison with ranked outputs could change how developers approach ambiguous tasks. The tradeoff is that it is still pre-release and compute-intensive.

Cursor feels like VS Code with superpowers. The codebase indexing means it understands your entire project from the start. Composer handles multi-file edits naturally. Background Agents let you kick off tasks and continue working. BugBot catches issues in PRs automatically. The tradeoff is vendor lock-in to their IDE (though it is VS Code-compatible).

Aider feels like a trustworthy pair programmer. Every change gets a git commit with a descriptive message, so you always have a clear history. It works with whatever model you prefer. The linting and test-running integration catches problems early. The tradeoff is no multi-agent capabilities and a more basic feature set.

6. Ecosystem

Skills, Plugins, and Extensions

Tool	Extension System	Notable Extensions	Community Size
Claude Code	Skills (SKILL.md), Plugins (npm), Hooks (lifecycle events), MCP servers	/deploy, /review-pr, /pipeline, /research, community skills repos	Large (fastest-growing)
Gemini CLI	MCP servers, Community tools, Agent Skills (preview)	Google Search, file ops, shell, web fetch, community MCP servers	Growing (open-source advantage)
Codex CLI	MCP servers	Web search, basic tool integrations	Medium
Grok Build	None yet	N/A	Pre-release
Cursor	VS Code extensions (full marketplace), .cursorrules, MCP	BugBot, Background Agents, all VS Code extensions	Very large (VS Code ecosystem)
Aider	MCP server (Aider itself is an MCP server), browser plugin	Playwright web scraping, linter integration, IDE plugins	Large (OSS community)

MCP (Model Context Protocol) Ecosystem

All major CLI tools now support MCP as a client, meaning they can connect to MCP servers for extended capabilities (databases, APIs, SaaS tools, etc.). This is a convergence point – the MCP ecosystem benefits all tools equally.

Tool	MCP Role	Configuration
Claude Code	Client	.claude/settings.json mcpServers
Gemini CLI	Client	settings.json mcpServers
Codex CLI	Client	Configuration file
Cursor	Client	.cursor/mcp.json
Aider	Server (exposes Aider as MCP server)	Via AiderDesk/WebSocket

7. Best For (Use Cases)

Decision Matrix

Use Case	Best Tool	Why
Complex multi-file refactors	Claude Code	Agent Teams parallelize subtasks; Opus 4.6 has highest first-pass correctness
Zero-budget development	Gemini CLI	1,000 free requests/day with Gemini 3 Flash is unbeatable
Security-sensitive codebases	Codex CLI	OS-native sandboxing (Seatbelt/Bubblewrap) provides strongest isolation
Exploring multiple solutions	Grok Build	8 parallel agents with Arena Mode ranking (when available)
Full IDE experience	Cursor	Deep codebase indexing + Composer + BugBot + Background Agents in a VS Code shell
Model flexibility / local LLMs	Aider	100+ model support including local models; no vendor lock-in
Remote/mobile development	Claude Code	Only tool with Remote Control (mobile apps, web interface)
CI/CD and automation	Claude Code	SDK/headless mode, hooks, /loop scheduling, Agent SDK
Quick bug fixes	Gemini CLI or Codex CLI	Fast, low-overhead, good enough for small changes
Open-source contribution	Aider or Gemini CLI	Fully open, community-driven, no proprietary dependencies
Team/enterprise deployment	Cursor or Claude Code	Business plans, admin controls, compliance features
Learning / teaching	Gemini CLI	Free tier removes cost anxiety; good documentation
Existing ChatGPT subscribers	Codex CLI	Included in ChatGPT Plus/Pro subscriptions
Monitoring / scheduled tasks	Claude Code	/loop is the only built-in scheduling primitive

Use Case	Best Tool	Why
Legacy codebase understanding	Cursor	Proprietary indexing engine handles massive monorepos

8. Benchmark Results

SWE-bench Verified (500 Python tasks, March 2026)

Rank	Model/Agent	Score	Notes
1	Gemini 3.1 Pro	80.6%	Google, released Feb 2026
2	Claude Opus 4.6 (Thinking)	80.8%	Anthropic, statistical tie with Gemini
3	GPT-5.4	78.2%	OpenAI, tie with GPT-5.3 Codex
4	GPT-5.3 Codex	78.0%	OpenAI, coding-optimized
5	Grok Code Fast 1	70.8%	xAI, Aug 2025 result

SWE-bench Pro (1,865 multi-language tasks)

This harder benchmark reveals the real gap between models. Tasks require an average of 107 lines of changes across 4.1 files.

Rank	Model	Score	Notes
1	GPT-5	23.3%	Best on hard tasks
2	Claude Opus 4.1	23.1%	Close second
-	Others	<20%	Significant dropoff

Key Insight: The gap between SWE-bench Verified (70-81%) and SWE-bench Pro (20-23%) shows that current AI coding agents excel at small, targeted fixes but still struggle with large, multi-file, multi-language changes. No tool has “solved” real-world software engineering.

SWE-CI (Continuous Integration, March 2026)

A new benchmark testing whether AI agents can maintain codebases over time without breaking existing functionality.

- **Finding:** 75% of tested AI coding agents break previously working code during long-term maintenance tasks, even when their initial patches pass all tests.
- **Implication:** The “first-pass” accuracy of a model is only part of the story. How well a tool preserves existing functionality matters enormously.

Real-World First-Pass Correctness (from developer reports)

Tool	First-Pass Success Rate	Notes
Claude Code	~92-95%	Highest reported; fewer iterations needed
Gemini CLI	~85-88%	Good but often needs one revision round
Codex CLI	~85-90%	Similar to Gemini; improving rapidly
Grok Build	Unknown	Pre-release; insufficient data
Cursor	~88-92%	Strong with codebase indexing context

Tool	First-Pass Success Rate	Notes
Aider	~80-90%	Varies significantly by model chosen

9. Pricing Comparison

Subscription Plans

Tier	Claude Code	Codex CLI	Cursor	Gemini CLI	Grok Build	Aider
Free	None	None	Yes (limited)	Yes (1,000 req/day)	Waitlist	Yes (BYOK)
~\$20/mo	Pro (\$20)	ChatGPT Plus (\$20)	Pro (\$20)	N/A	N/A	N/A
~\$60/mo	N/A	N/A	Pro+ (\$60)	N/A	N/A	N/A
~\$100/mo	Max 5x (\$100)	N/A	N/A	N/A	N/A	N/A
~\$150/mo	Business (\$150/seat)	N/A	N/A	N/A	N/A	N/A
~\$200/mo	Max 20x (\$200)	ChatGPT Pro (\$200)	Ultra (\$200)	N/A	N/A	N/A
API / Pay-as-you-go	Yes	Yes	N/A	Yes (API key)	Yes (\$0.20/\$1.50 per M tokens)	Yes (any provider)

Cost Per Task (Estimated, API pricing)

Task Type	Claude Code	Gemini CLI	Codex CLI	Grok Build
Simple bug fix	\$0.15-0.40	\$0.05-0.15	\$0.10-0.30	\$0.02-0.10
Feature implementation	\$0.80-2.50	\$0.30-1.00	\$0.50-1.50	\$0.10-0.50
Large refactor	\$3.00-12.00	\$1.00-5.00	\$2.00-8.00	\$0.50-3.00
Multi-file migration	\$5.00-20.00	\$2.00-8.00	\$3.00-12.00	\$1.00-5.00

Note: Claude Code tends to cost less per *completed* task despite higher per-token pricing because it requires fewer iterations. Grok Build offers the lowest per-token pricing. Gemini CLI’s free tier covers most individual developer needs at zero cost.

Best Value by Profile

Developer Profile	Recommended Plan	Monthly Cost
Hobbyist / student	Gemini CLI free tier	\$0
Individual developer (moderate use)	Claude Code Pro or ChatGPT Plus + Codex	\$20
Professional developer (heavy use)	Claude Code Max 5x	\$100
Power user / team lead	Claude Code Max 20x or Cursor Ultra	\$200
Enterprise team	Claude Code Business or Cursor Business	\$150-200/seat
Budget-conscious professional	Aider + Gemini Flash API	\$5-15 (API only)
Cost-optimized high-volume	Grok Build API (when available)	Variable (lowest per-token)

10. Verdict & Recommendations

The Clear Winner (Overall): Claude Code

Claude Code is the most complete AI coding CLI tool available in March 2026. No other tool matches its combination of: - Highest first-pass code correctness (~92-95%) - Deepest feature stack (Skills, Hooks, Plugins, Memory, Agent Teams, Remote Control, Voice, Computer Use, Scheduling) - Most mature ecosystem (14 months of iteration since launch) - Best autonomy features (Background Agents, /loop, Agent Teams) - Only tool with mobile access via Remote Control

Weaknesses to acknowledge: Highest per-token API cost. No free tier. Source-available but not fully open source. The sheer number of features can be overwhelming for new users.

The Best Free Option: Gemini CLI

Gemini CLI's free tier (1,000 requests/day on Flash, access to Pro) is genuinely useful for real development work, not just a trial. Combined with being fully open source (Apache 2.0), having a 1M context window, and Google Search grounding, it is the no-brainer choice for anyone who cannot or does not want to pay for an AI coding subscription.

Weaknesses to acknowledge: Fewer agent orchestration features. Lower first-pass correctness than Claude Code. Still catching up on the feature roadmap.

The Safest Execution: Codex CLI

If your primary concern is security and you want the strongest sandbox guarantees, Codex CLI's OS-native sandboxing (Seatbelt on macOS, Bubblewrap on Linux) combined with its Rust implementation provides the highest confidence that generated code will not harm your system. The Rust rewrite also makes it the fastest-starting CLI tool.

Weaknesses to acknowledge: Smaller extension ecosystem. 272K standard context window (1M is experimental). Thinner customization options compared to Claude Code.

The Most Promising: Grok Build

Grok Build's 8-parallel-agent architecture with Arena Mode is genuinely novel and could change how developers approach complex, ambiguous problems. Combined with the lowest per-token pricing and 2M context windows, it has the ingredients to be a serious contender. The question is purely one of availability and maturity.

Weaknesses to acknowledge: Still on waitlist. No public track record. Unclear feature depth beyond parallel agents. Infrastructure demands may limit scaling.

The IDE Developer's Choice: Cursor

If you prefer working inside an IDE rather than a terminal, Cursor is the clear leader. Its codebase indexing is unmatched, Composer 2.0 handles multi-file edits naturally, Background Agents run in the cloud, and BugBot provides automated PR reviews. The VS Code extension ecosystem gives it the widest plugin compatibility.

Weaknesses to acknowledge: IDE-only (no terminal-first workflow). Closed source. Monthly subscription required. Not suitable for headless/CI/CD automation.

The Model-Agnostic Veteran: Aider

If you want no vendor lock-in and the ability to use any model (including local LLMs), Aider is your tool. Its git integration is the best of any tool – every AI change is a separate commit with a descriptive message. It costs nothing beyond API fees. And it works with 100+ models across every major provider.

Weaknesses to acknowledge: No multi-agent capabilities. No background agents. Simpler feature set. Quality depends entirely on which model you choose.

Quick Decision Guide

START

Do you need a free option?
Yes, fully free Gemini CLI (free tier)
Yes, but I'll pay for API calls Aider (BYOK)

Do you prefer an IDE over a terminal?
Yes Cursor

Is execution security your top priority?
Yes Codex CLI

Do you want maximum capability?
Yes Claude Code

Do you want model flexibility / local LLMs?
Yes Aider

Do you want to explore parallel agents?
Yes (and can wait) Grok Build (waitlist)

Methodology & Disclaimers

- All information is current as of March 25, 2026.
 - Benchmark scores are sourced from official announcements and third-party verification (Epoch AI, Scale Labs).
 - Pricing reflects published rates; actual costs vary by usage patterns and caching behavior.
 - “First-pass correctness” estimates are aggregated from developer reports and blog comparisons, not controlled studies.
 - Grok Build information is based on pre-release reports and xAI announcements; real-world performance may differ.
 - This comparison aims to be fair and factual. Each tool has legitimate strengths and appropriate use cases.
-

Sources

- [Claude Code Changelog](#)
- [Claude Code March 2026 Updates](#)
- [Claude Code Docs - Memory](#)
- [Claude Code Docs - Skills](#)
- [Claude Code Docs - Hooks](#)
- [Gemini CLI GitHub Repository](#)
- [Google Blog - Introducing Gemini CLI](#)
- [Gemini 3.1 Pro on Gemini CLI](#)
- [Gemini CLI Session Management](#)
- [Codex CLI Features - OpenAI Developers](#)
- [Codex CLI Reference](#)
- [OpenAI - Introducing Codex](#)
- [Codex CLI Rust Rewrite Discussion](#)
- [GPT-5.4 Introduction](#)
- [Grok Build - xAI CLI Coding Agent](#)
- [Grok Code Fast 1 - xAI](#)
- [xAI Models and Pricing](#)
- [Cursor Features](#)
- [Cursor Pricing](#)
- [Cursor 2.0 Multi-Agent Architecture](#)
- [Aider Documentation](#)
- [Aider GitHub Repository](#)

- [SWE-bench Verified Leaderboard](#)
- [SWE-bench Pro Leaderboard](#)
- [SWE-bench February 2026 Update](#)
- [Claude Code vs Codex vs Gemini - Educative](#)
- [Gemini CLI vs Claude Code vs Codex - Inventive HQ](#)
- [OpenAI Codex Pricing Analysis](#)
- [AI Coding Tools Comparison 2026 - NxCode](#)
- [Best AI for Coding 2026 - Morph LLM](#)